Health Retailers – Dataset Guide

What every CSV file contains, how good it is, how the tables connect, and how the data serves the Data Engineering project goals.

Class 3IA · Data Engineering Project – Project 2: Health retailers
Source folder: `projet/fichiers/fichiers` · 29 CSV files · period covered: 30 Sep 2024 → Apr 2025

> **In one paragraph.** The files are a raw database export from the point-of-sale / ERP software of one drugstore (parapharmacie). They cover the product catalogue, customers and suppliers, every sale and purchase document with its lines, payments and cash registers, and every stock movement. About **10,300 counter sales** (≈ **715,000** in TTC value), **38,700 sold lines**, **4,580 distinct products sold** out of a catalogue of ~7,800, bought from 118 suppliers.

> **Context to mention in your report.** The software was built for Tunisia (currency symbol "DT", payment mode `Cash.Tunisia`, VAT rates 7 / 13 / 19 %, Tunis address in the invoice template). Customer and supplier names were then anonymised and relocated to France (Paris, Marseille, Toulouse, `example.com` e-mails). Treat amounts as a single currency and state this assumption.

Contents

1. Project goals (from the presentation)
2. Summary of all files – keep / drop verdict
3. File-by-file description and improvements
4. How the tables connect (diagram + join keys)
5. Global data-quality problems and the cleaning plan
6. Map to the project goals: data warehouse, KPIs, clustering, scoring, prediction

1. Project goals (from `PrésentationDE.pdf`)

Project 2 asks for a **decision-support and analytics platform** combining Business Intelligence, Data Engineering and AI, to centralise, transform, analyse and add value to the transactional data of a drugstore located in France. The presentation frames the work in phases:

Phase	What is expected	Graded skill
Business & data understanding	Understand the business and what every data source means (this document).	—
Data collection & preparation (ETL)	Extract the raw files, clean / map / transform them (filter, aggregate, join, convert types), load them into a data warehouse.	Python for Data Engineering · Data Warehousing
Data analysis & ML	**Clustering** (profiles by similarity and behaviour), **Scoring**, **Prediction** (business opportunity, commercial risk, time series).	Machine Learning basics
Reporting / visualisation	Dashboards with dynamic KPIs, drill-down by time; optionally a web or mobile application.	Data Warehousing

2. Summary of all files

CORE fact / main data **USEFUL** dimension or enrichment **LOW** optional, niche use **DROP** not needed for analysis

File	Rows	Area	Verdict	Role in the warehouse
documentItemm.csv	38,705	Sales/purchases	CORE	Grain of `Fact_Sales` / `Fact_Purchases` (one row per product line)
Documents.csv	12,542	Sales/purchases	CORE	Document header: date, type, user, payment mode, totals
StockMovements.csv	49,609	Stock	CORE	`Fact_StockMovement`; purchase price and average cost (CUMP)
Produits - ….csv	7,800	Products	CORE	`Dim_Product`: family, brand, purchase / sale prices
Transactions.csv	12,523	Payments	CORE	Bridge document ↔ customer/supplier ↔ payment; paid / unpaid status
PaymentsList.csv	10,280	Payments	CORE	`Fact_Payment` with readable payment-method names
ProductStocks.csv	6,842	Stock	USEFUL	Current stock snapshot; **key table** linking lines to products
DocumentTypes.csv	15	Reference	USEFUL	`Dim_DocType` – required to separate sales from purchases
ProductBrands.csv	212	Products	USEFUL	`Dim_Brand`
Clients - ….csv	93	Partners	USEFUL	`Dim_Customer` (city, balance)
Fournisseurs - ….csv	118	Partners	USEFUL	`Dim_Supplier`
CompaniesList.csv	156	Partners	USEFUL	Maps numeric `Company_Id` → customer / supplier code
TransactionPayments.csv	10,962	Payments	USEFUL	Bridge transaction ↔ payment
ProductPrices.csv	14,756	Products	USEFUL	Price lists and profit margin per product

InventaireItems.csv	6,777	Stock	**USEFUL**	Theoretical vs counted stock (shrinkage KPI)
DomainEvents.csv	21,743	Treasury	**USEFUL**	Register opening/closing, cash discrepancies
CaisseOperations.csv	9,707	Treasury	**USEFUL**	Cash-register movements per register / user
TresoreriePayments.csv	10,280	Treasury	**USEFUL**	Which register or bank account received each payment
users_csv.csv	7	Reference	**USEFUL**	`Dim_Employee` (drop the password columns)
Barcodes.csv	6,444	Products	**LOW**	EAN code – useful only for joining with external product data
ItemTaxes.csv	37,707	Taxes	**LOW**	Duplicates the VAT already stored on each line
DocumentsReports.csv	11,975	Sales	**LOW**	JSON flags per line; little analytical value
RelatedDocuments.csv	864	Sales	**LOW**	Links between documents (order → delivery → invoice)
Events.csv	82,585	Audit	**LOW**	Audit log; overlaps with StockMovements
taxRates.csv	28	Taxes	**LOW**	Decode a tax-rate id if needed
Payment.csv	10,899	Payments	**DROP**	Duplicate of PaymentsList (10,168 common ids)
ApplicableTaxes.csv	6,428	Config	**DROP**	Software tax configuration
Permissions.csv	104	Config	**DROP**	User access rights
users_settings_csv.csv	95	Config	**DROP**	Application settings (contains an embedded logo image)

3. File-by-file description and improvements

A. Products

Produits - 24112025081625 (2).csv

CORE 7,800 rows · 15 cols · ";"

Product catalogue exported from the software screen (French headers). One row per product.

MAIN COLUMNS

`Code` (GUID), `Libellé` (name), `Famille`, `Marque`, `Prix d'achat HT`, `Prix de vente HT`, `Tva`, `Prix de vente TTC`, `Quantité`, `Unité`, `Quantité d'Alerte`.

OBSERVATIONS

- Families are very unbalanced: TOILETTE ≈ 7,029, COMPLEMENTS ALIMENTAIRES 237, ACCESSOIRE 32, COMPRIME 15, ANIMALERIE 11, and a few with 1–3 products.
- 465 products have no family; only 3,595 of 7,800 have a brand.
- **67 duplicated codes.**
- **No numeric `Id`**: this file cannot be joined directly to `ProductStocks.Product_Id`. The only common key with sales lines is the **product name** (83 % of sold lines match exactly after cleaning).
- Accented characters are already corrupted in the file (`Libell`, `PUR`).

IMPROVEMENTS

- Rename columns to snake_case English, fix the broken accents (→ é/è).
- Deduplicate on `Code`; replace empty family by "NON CLASSÉ"; merge tiny families (XEN, NATURE, POMMADE…) into "AUTRES".
- Build a normalised name key (upper-case, trimmed, single spaces) to join with sales lines; then compute **unit margin** = sale HT − purchase HT and **margin rate**.
- Drop the empty columns `Libellé ticket de caisse`, `Poids (Kg)`, `Observations`.

ProductStocks.csv

USEFUL 6,842 rows · 5 cols · ";"

Current stock quantity per product. Columns: `Id`, `Quantity`, `Stock_Id` (always 1 – one store), `Product_Id`, `TotalQuantity`.

WHY IT MATTERS

It is the **hinge** of the model: sales lines and stock movements point to `ProductStock_Id`, and barcodes / prices point to `Product_Id`. In practice `Id` = `Product_Id` here.

IMPROVEMENTS

- 132 products have a **negative stock** (sold without being received): flag them as a data-quality issue rather than delete them.
- `Quantity` and `TotalQuantity` are redundant (single store) – keep one.

ProductBrands.csv

USEFUL 212 rows · 2 cols · ","

List of brands (`Id`, `BrandName`: BIOHERBS, TITANIA…).

IMPROVEMENTS

- No product table references a brand id: join to `Produits.Marque` by **name** (trim + upper-case). Unify spelling variants.

Barcodes.csv

LOW 6,444 rows · 4 cols · ";" · Latin-1

EAN-13 barcode per product (`Product_Id`, `Code`, `Type`).

IMPROVEMENTS

- Read `Code` as **text** (not a number – otherwise leading zeros and precision are lost). Drop code "0".
- Optional enrichment: the EAN allows matching with public product databases (e.g. Open Beauty Facts) to add categories or ingredients.

ProductPrices.csv

USEFUL 14,756 rows · 9 cols · ";"

Price lists: `PriceHt`, `PriceTtc`, `UseTva`, `ProfitMargin`, `Product_Id`, `PriceName_Id` (≈ 2 price lists per product).

IMPROVEMENTS

- Comma decimals (`289,92`) → dot; `VRAI/FAUX` → boolean.
- Keep only the retail price list (`PriceName_Id = 1`, the one used by counter sales) for `Dim_Product`; drop the empty `ForcedPriceTtc`, `PrixTvc`.
- `ProfitMargin` is often empty – recompute it from purchase and sale prices.

B. Customers, suppliers, users

Clients - 24112025081756 (3).csv

USEFUL 93 rows · 11 cols · ";"

Named customers: `Code` (CL0001…), `Nom`, address, postcode, city, e-mail, `Solde` (balance owed), `Solde initial`.

OBSERVATIONS

- Two kinds: 46 "Client particulier" (individuals) and 47 "Laboratoire" (businesses). The type is hidden inside the name: `"1. Client particulier – Julien Moreau"`.
- Most counter sales are **not** attached to a named customer – see CompaniesList below.

IMPROVEMENTS

- Split `Nom` into `customer_type` and `customer_name`; remove the "1." numbering; trim every text field (many trailing spaces).
- Drop `Site Web` and `Registre de commerce` (empty). Keep `Ville` for geographic analysis.
- Remove e-mails from the analytical warehouse (personal data – GDPR good practice).

Fournisseurs - 24112025081816 (2).csv

USEFUL 118 rows · 10 cols · ";"

Suppliers (laboratories and distributors): `Code` (FOU0001…), name, address, city, e-mail, `Solde`.

IMPROVEMENTS

- Same cleaning as Clients (remove numbering, trim). Used for `Dim_Supplier` and supplier analyses (purchase volume, dependency).

CompaniesList.csv

USEFUL 156 rows · 18 cols · ";" · Latin-1

Technical table giving the numeric `Company_Id` used by Transactions and payments, with `CompanyType_Id` (1 = customer, 2 = supplier) and the `Code` (CL…/FOU…).

OBSERVATIONS

- **Malformed**: the name column header is `| Name |` and values are wrapped in Markdown pipes; 10 empty trailing columns.
- **Company 2 = the generic "walk-in customer"** (its name was lost and shows as `|------|`). It receives **8,894 of 12,523 transactions (71 %)**.

IMPROVEMENTS

- Strip the `|` characters, drop `Id3…Id7` and the empty columns; set company 2 name = "Client comptoir (anonyme)".
- Use `Code` to attach the Clients / Fournisseurs details.

users_csv.csv

USEFUL 7 rows · 6 cols · ";"

Staff accounts: Administrateur, Système, Marie, Hugo, Celine, melanie, kady. Referenced by `CreatedBy`, `ValidatedBy`, `User_Id`, `AddedByUserId`.

IMPROVEMENTS

- **Delete `Password` and `PasswordSalt`** (security). Keep `Id` + `FullName` as `Dim_Employee`; flag admin/system as non-sales users.

C. Sales and purchase documents

DocumentTypes.csv

USEFUL 15 rows · 16 cols · ";"

Reference that explains `DocumentType_Id`. **Read this before any analysis**, otherwise sales and purchases get mixed.

Id	Name	Meaning	Stock	Docs
6	VenteComptoir	Counter sale (till receipt)	−1	10,314
5	BonLivraison	Delivery note to a customer	−1	1,032
20	FactureFour	Supplier invoice (purchase)	+1	883
1	FactureClient	Customer invoice	−1	249
2 / 4 / 7 / 23 / 3	Proforma, AvoirClient, BonRetour, AvoirFour, CommandeClient	Quote, customer credit note, return, supplier credit note, order	0 / ±1	< 30 each

IMPROVEMENTS

- Keep `Id`, `Name`, `StockFactor`, `IsSellType`, `IsRefund`; add an English label and a `flow` column (SALE / PURCHASE / RETURN / QUOTE). Drop `IconPath`, `ReportBottomMsg`.

Documents.csv

CORE 12,542 rows · 35 cols · "," · Latin-1

Header of every document: `DocumentNumber` (VC0001/24…), `DocumentDate`, `State` (1 = validated, 0 = draft – 56 drafts), `TotalHt`, `TotalNetHt`, `TotalTva`, `TotalTtc`, `FinalPrice`, `DocumentType_Id`, `Transaction_Id`, `PaymentMode_Id`, `CreatedDate/By`, `ValidatedDate/By`.

IMPROVEMENTS

- Dates are in **US format** (`9/30/2024 0:00`) – parse with `%m/%d/%Y`. `DocumentDate` has no time: use `CreatedDate` for the hour of the sale (hourly traffic analysis).
- Exclude drafts (`State = 0`) and non-financial types (Proforma, CommandeClient) from revenue.
- Drop 5 empty columns (`DiscountPercentage`, `Observation`, `UserComment`, `Attachments_Id`, `DiscountAmount`) and the retention columns (always 0). Parse `Extra` JSON (`cosld` = customer balance) or drop it.

documentItemm.csv

CORE 38,705 rows · 34 cols · ";"

The most important file: one row per product line of a document. `Document_Id`, `ProductStock_Id`, `Description`, `Quantity`, `PrixHt`, `Tva`, `DiscountPourcentage`, `PrixTotalHt`, `PrixTotalTtc`, `TotalTva`, `PrixTtc` …

OBSERVATIONS

- `Description` contains **HTML** (`<p style=…><span…>NAME</span></p>`).
- Many `Sum…` columns repeat the line totals with more decimals; 11 columns are empty.
- Discounts are frequent (e.g. 20 %) – interesting for promotion analysis.

IMPROVEMENTS

- Strip HTML tags and ` ` to recover the product name (then use it to join with Produits).
- Comma decimals → dot. Keep one set of amounts (`Quantity`, `PrixHt`, `DiscountPourcentage`, `PrixTotalHt`, `TotalTva`, `PrixTotalTtc`); drop the `Sum…` duplicates and empty columns.
- Add `unit_cost` (purchase price from Produits or `cump` from StockMovements) → `line_margin`.

ItemTaxes.csv

LOW 37,707 rows · 14 cols · ","

VAT detail per document line (`DocumentItem_Id`, rate, base, amount). The same values already exist in `documentItemm.Tva` / `TotalTva`.

IMPROVEMENTS

- Use only to check that line VAT is consistent; otherwise not needed in the warehouse.

DocumentsReports.csv

LOW 11,975 rows · 2 cols · ";"

`Id` + one JSON column `Benef` with, per line, the product id and two technical flags (`bcdde`, `bcdve`). Despite its name it does not hold profit amounts.

IMPROVEMENTS

- Can be dropped; the margin is better computed from prices and costs.

RelatedDocuments.csv

LOW 864 rows · 4 cols · ";"

Pairs `Document1_Id` ↔ `Document2_Id` (e.g. delivery notes grouped into an invoice). `Relation` is always 1.

IMPROVEMENTS

- Use it to **avoid double counting**: a delivery note and the invoice that bills it represent the same sale. Otherwise optional.

D. Transactions and payments

Transactions.csv

CORE 12,523 rows · 12 cols · ";"

One business transaction per sale or purchase: `IsPayed`, `PayedAmount`, `IsDestocked`, `Company_Id` (customer/supplier), `PaymentDocument_Id`, `DestockDocument_Id`.

IMPROVEMENTS

- This is where the **customer** of a sale is found (`Documents.Transaction_Id → Transactions.Company_Id`).
- `IsPayed = FAUX` gives unpaid documents → basis for a **commercial-risk** / credit analysis.
- Drop the 4 empty columns (refund, restock, billing and shipping addresses).

TransactionPayments.csv

USEFUL 10,962 rows · 4 cols · ","

Bridge table: `Transaction_Id` ↔ `Payment_Id` with the amount allocated. One sale can be paid with several methods (e.g. part card, part cash).

PaymentsList.csv

CORE 10,280 rows · 10 cols · ";"

Payments with `Amount`, `PaymentDate`, `Company_Id`, `PaymentMode_Id`, `PaymentMethodName` (BankCard, Cheque, Cash.Tunisia, Credit, Jetons) and `Extra` JSON (`gA` = cash given, `rA` = change returned).

IMPROVEMENTS

- `PaymentMethodName` is filled for only ~620 rows: rebuild it for every row from `PaymentMode_Id` (e.g. 5 = BankCard, 11 = Cash).
- Dates are **DD/MM/YYYY** (different from Documents). Remove `IsRemoved = VRAI` rows.
- Rename `Cash.Tunisia` → `Cash`.

Payment.csv

DROP 10,899 rows · 9 cols · ";" · Latin-1

Same content as PaymentsList (10,168 ids in common) but without the method name and with broken accents. Ids 1–1,761 exist only here (the first days).

IMPROVEMENTS

- Use PaymentsList as the main source and append only the ~730 ids missing from it; then discard this file.

E. Treasury (cash registers and bank)

TresoreriePayments.csv

USEFUL 10,280 rows · 11 cols · ","

For each payment (`SalePaymentId` → PaymentsList.Id): `Amount`, `CaisseId` (register) or `BankAccountId`, `IsPurchase`, `CreatedAtUtc`, `CreatedBy`, `Removed`.

IMPROVEMENTS

- Filter `Removed = TRUE`. Map the 6 register GUIDs to readable names (Caisse 1…6).

CaisseOperations.csv

USEFUL 9,707 rows · 12 cols · ","

Each movement in a cash register: `CaisseId`, `Amount`, `AddedAtUtc`, `AddedByUserId`, `Status`, cancellation fields. Two registers carry almost everything (5,177 and 4,458 operations).

IMPROVEMENTS

- Its `TransactionId` / `PaymentId` GUIDs do **not** match any other table – it can only be joined on `CaisseId` + user + timestamp. Use it for register-level KPIs (volume per register, per cashier, per hour).

DomainEvents.csv

USEFUL 21,743 rows · 8 cols · "," · Latin-1

Event log of the treasury module: PaymentAdded (10,280), CaisseTransactionAdded (9,707), BankAccountTransactionAdded (521), **CaisseDiscrepancyIdentified (437)**, CaisseOpened / Closed (~263), PaymentRemoved (112)…

OBSERVATIONS / IMPROVEMENTS

- **Malformed**: every whole line is wrapped in quotes, so a normal CSV reader sees only one column. Remove the outer quotes and un-double the inner ones before parsing; then parse `Data` as JSON.
- Keep only CaisseOpened / Closed / DiscrepancyIdentified → KPI "cash discrepancy per register / cashier", and register opening hours.

F. Stock and audit

StockMovements.csv

CORE 49,609 rows · 11 cols · ";"

Every stock change: `ProductStock_Id`, `DocumentId`, `MovementDate`, `Quantity` (+ in / − out), `CurrentBalance`, `Description`, `User_Id`, `Extra` JSON with `prc` (purchase price) and `cump` (weighted average unit cost).

OBSERVATIONS

- Reasons: Validation de Vente (19,669), Validation de Facture (9,972), Validation de Bon (7,881), Stock initial (6,842), Corrections (~4,900), manual changes (248).

IMPROVEMENTS

- Extract a `movement_type` from `Description` (SALE, PURCHASE, DELIVERY, INITIAL, CORRECTION, MANUAL) and the document number.
- Parse `Extra` → `cump` is the **best cost price** for margin calculation (it changes over time).
- Rebuild a **daily stock level per product** → stock-out days, rotation, coverage.

InventaireItems.csv

USEFUL 6,777 rows · 9 cols · ","

Physical inventory: `TheoricalQuantity` (system) vs `FinalRealQuantity` (counted), per `Product_Id`.

IMPROVEMENTS

- `FinalRealQuantity` is almost always empty (> 95 %) – the count was not finished. Only usable as a stock snapshot; mention the limitation.

Events.csv

LOW 82,585 rows · 12 cols · ";" · 20 MB

General audit log: `EntityId` (e.g. product stock id), `EventType`, JSON `Data` (`qte`, `tQte`), `Reason` ("Validation de Vente Comptoir – VC0001/24"), `Timestamp`, `UserId`.

IMPROVEMENTS

- Mostly repeats StockMovements. Use it only for "who did what" (user activity) analysis; otherwise skip it to save load time.

G. Configuration files

taxRates.csv

LOW 28 rows

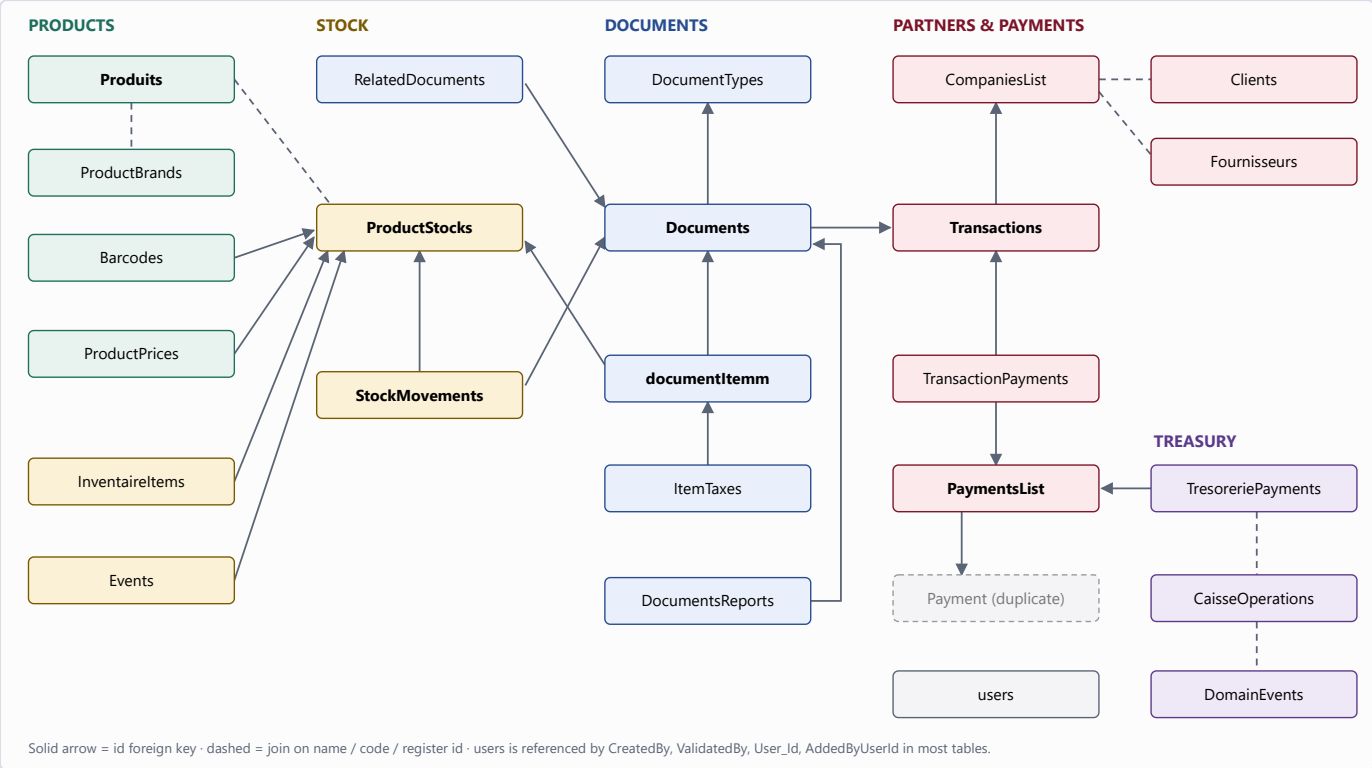
Tax definitions by country and period (7 %, 13 %, 19 % VAT; stamp duty…). Useful only to translate a `TaxRate_Id`; the rate is already on each line.

ApplicableTaxes.csv · Permissions.csv · users_settings_csv.csv DROP

Software configuration: which tax applies to which product/document, user permission GUIDs, and application settings (currency symbol “DT”, default alert quantity 5, invoice template with an embedded base64 logo). No analytical value – keep only as evidence of the Tunisian origin of the software.

4. How the tables connect

Arrows go from the table holding the foreign key to the table it refers to. Dashed lines are joins on **text** (name or code), not on an id.



Join keys

From (foreign key)	To	Type	Note
`documentItemm.Document_Id`	`Documents.Id`	id	line → document header
`documentItemm.ProductStock_Id`	`ProductStocks.Id`	id	line → product
`ProductStocks.Product_Id`	`Barcodes.Product_Id`, `ProductPrices.Product_Id`, `InventaireItems.Product_Id`	id	numeric product id
clean(`documentItemm.Description`)	`Produits.Libellé`	**name**	only way to get family / brand / purchase price (~83 % match)
`Produits.Marque`	`ProductBrands.BrandName`	**name**	trim + upper-case
`Documents.DocumentType_Id`	`DocumentTypes.Id`	id	sale / purchase / return
`Documents.Transaction_Id`	`Transactions.Id`	id	document → transaction
`Transactions.Company_Id`	`CompaniesList.Id`	id	2 = anonymous walk-in customer
`CompaniesList.Code`	`Clients.Code` / `Fournisseurs.Code`	**code**	CL… = customer, FOU… = supplier

`TransactionPayments.Transaction_Id` / `Payment_Id`	`Transactions.Id` / `PaymentsList.Id`	id	bridge (n–n)
`TresoreriePayments.SalePaymentId`	`PaymentsList.Id`	id	payment → register / bank
`TresoreriePayments.CaisseId`	`CaisseOperations.CaisseId`, `DomainEvents.AggregateId`	register GUID	6 registers
`StockMovements.ProductStock_Id` / `DocumentId`	`ProductStocks.Id` / `Documents.Id`	id	DocumentId = 0 for initial stock
`ItemTaxes.DocumentItem_Id`	`documentItemm.Id`	id	
`RelatedDocuments.Document1_Id` / `Document2_Id`	`Documents.Id`	id	
`*.CreatedBy, ValidatedBy, User_Id, AddedByUserId`	`users.Id`	id	cashier / employee

5. Global data-quality problems and cleaning plan

Problem	Where	Fix in the ETL
Two delimiters	";" in 20 files, "," in 9 files	Detect the delimiter per file (or declare it in a config dict).
Mixed encodings	Barcodes, CompaniesList, Documents, DomainEvents, Payment are Latin-1; others UTF-8 with BOM; Produits already contains `&#xFFFD;`	Read with `encoding='utf-8-sig'`, fall back to `latin-1`; repair known broken words.
Decimal comma	`42,38` in Payment, PaymentsList, ProductPrices, Transactions, documentItemm	`decimal=','` when reading, or replace before casting to float.
Two date formats	US `9/30/2024` (Documents, CaisseOperations, TresoreriePayments) vs FR `30/09/2024` (StockMovements, Events, PaymentsList)	Parse each file with an explicit format; store as ISO datetime; build `Dim_Date`.
Two boolean styles	`VRAI/FAUX` vs `TRUE/FALSE`	Map both to `True/False`.
HTML in text	`documentItemm.Description`	Remove tags with a regex, unescape entities.
JSON in columns	`Extra`, `Data`, `Benef`	`json.loads`, keep only the needed keys (`cump`, `prc`, `gA`, `rA`).
Malformed files	CompaniesList (Markdown pipes), DomainEvents (quoted whole lines)	Custom pre-processing before the CSV parser.
Empty columns	≈ 50 columns are > 95 % empty	Drop them automatically (threshold rule).
Duplicates	67 product codes; Payment vs PaymentsList	Deduplicate; keep a single payment source.
Missing links	Produits has no numeric id; 465 products without family; 71 % of sales with anonymous customer	Name-based join with a match-rate check; "NON CLASSÉ" family; treat walk-in customer separately.
Inconsistent values	132 negative stocks; 56 draft documents; cancelled/removed payments	Flag, filter drafts and removed rows, document every rule.
Sensitive data	Password hashes, e-mails, addresses	Drop from the warehouse.
Trailing spaces & numbering	Clients, Fournisseurs ("1. Client particulier — …")	Trim, split type and name.

> **Suggested ETL layers. Bronze** – raw CSVs loaded as-is (text columns) · **Silver** – cleaned, typed, deduplicated tables with English names · **Gold** – star schema (facts + dimensions) used by dashboards and ML. Tools: Python (pandas) for extraction and cleaning, PostgreSQL / SQL Server for the warehouse, Power BI for dashboards.

6. Map to the project goals

6.1 Data warehouse – star schema

Table	Grain / content	Built from
Fact_Sales	One row per sold product line: qty, unit price HT, discount %, total HT, VAT, total TTC, unit cost, margin	documentItemm + Documents (types 6, 5, 1; minus 4, 7 as returns) + StockMovements (cump)
Fact_Purchases	One row per purchased line: qty, cost, supplier	documentItemm + Documents (types 20, 22, 23, 24)
Fact_StockMovement	One row per movement: qty in/out, balance, cost	StockMovements
Fact_Payment	One row per payment: amount, method, register, cashier	PaymentsList + TransactionPayments + TresorerriePayments
Fact_CashSession (optional)	Register opening/closing and discrepancies	DomainEvents + CaisseOperations
Dim_Date	Day, week, month, quarter, weekday, holiday, hour (separate Dim_Time)	generated
Dim_Product	Name, family, brand, barcode, purchase / sale price, alert quantity	Produits + ProductStocks + Barcodes + ProductPrices + ProductBrands
Dim_Customer / Dim_Supplier	Code, name, type, city	CompaniesList + Clients / Fournisseurs
Dim_Employee	Cashier name	users_csv
Dim_DocType · Dim_PaymentMode · Dim_Register	Small reference dimensions	DocumentTypes · PaymentsList · TresorerriePayments

6.2 Reporting – dashboard KPIs

Sales performance

- Revenue HT/TTC by day, week, month (trend Sep 2024 → Apr 2025)
- Number of tickets, average basket (value and items)
- Top / flop products, families, brands
- Sales by hour and weekday (store traffic)
- Sales by cashier

Profitability

- Gross margin and margin rate by product / family / brand
- Impact of discounts on margin
- Purchases vs sales by supplier

Stock

- Stock value, rotation, days of coverage
- Products under alert quantity / out of stock / negative
- Dormant products (no sale for N days)

Cash & payments

- Payment-method mix (card, cheque, cash, credit)
- Unpaid amounts and customer balances
- Cash discrepancies per register and cashier

6.3 Clustering

The presentation asks for customer profiles by similarity and behaviour. Because 71 % of sales are anonymous and only 93 customers are named, adapt it:

Clustering target	Features	Method	Business use
Products (main)	Sales volume, revenue, margin rate, sales frequency, price, discount rate, stock rotation	K-Means / hierarchical + ABC-XYZ	Stars, cash cows, slow movers, products to delist
Baskets / tickets	Basket value, number of items, families present, hour, weekday, payment method	K-Means / DBSCAN	Shopping missions (quick purchase, beauty basket, supplements…)
Named customers	RFM: recency, frequency, monetary + type (individual / laboratory), balance	RFM scores + K-Means	Loyal, at risk, B2B customers
Association rules (bonus)	Products bought together in the same ticket	Apriori / FP-Growth	Cross-selling, shelf placement, bundles

6.4 Scoring

- **Product score** (0–100): weighted mix of revenue, margin, rotation and trend → which products to push or reorder.
- **Customer credit-risk score**: from balances (`Solde`), unpaid transactions (`IsPayed = FAUX`), payment by credit/cheque → commercial risk.
- **Supplier score**: purchase volume, share of sales generated, margin of their products.

6.5 Prediction

Prediction	Data	Models
Sales forecasting (daily / weekly revenue, per family)	Fact_Sales aggregated by day (~7 months)	Prophet, SARIMA, XGBoost with date features; LSTM as a comparison
Demand per product → reorder	Daily quantity sold per product + current stock	Moving-average / XGBoost; derive reorder point and stock-out date
Stock-out risk (classification)	Stock level, sales velocity, supplier delay (from purchase documents)	Logistic regression / Random Forest
Business opportunity	Association rules + basket clusters	"Next product" recommendation
Commercial risk	Customer balances, unpaid history	Classification of customers at risk of non-payment

> **Limitation to state honestly.** Only ~7 months of history (no full year) → seasonality (summer, Christmas) can be observed only once or not at all. Keep forecasts short-horizon (weeks) and validate on the last month (April 2025).

6.6 Suggested workflow

1. Load all CSVs into a Bronze schema (Python script, one config entry per file: delimiter, encoding, date format).
2. Clean into Silver (rules of section 5), with a data-quality report (row counts before/after, match rates).
3. Build the Gold star schema and `Dim_Date`.
4. Dashboards in Power BI on Gold.
5. ML notebooks (clustering, scoring, forecasting) reading from Gold; write results back as tables (e.g. `product_cluster`, `sales_forecast`) so they appear in the dashboards.
6. Optional: a small web app (Streamlit / Flask) exposing KPIs and predictions.